Build the rental inventory.

Vue frontend + supplied PHP API + XAMPP database

GearDesk rents equipment for meetings and events. Staff need to manage equipment names, categories, daily rental rates, and available quantities from one responsive screen.

Assessment window	Your responsibility
5 h 30 min total	30 min preparation/downloads + 5 h implementation for this rehearsal.
Provided materials	SQL database, PHP API, backend .env, API documentation, reference mockups.
Candidate output	Vue application, responsive styling, API integration, validation, tests, plan, README, Git history.

Client requirements

- Load records automatically and display daily rates in pesos with two decimal places.
- Create and edit all four fields in modals. Populate category choices from the API.
- Confirm deletion in a modal and handle an empty HTTP 204 response.
- Use server-side pagination: five records per page. Read page totals from the API.
- Support name search and category filtering, with Apply and Clear actions.
- Provide visible loading, success, empty, validation, and failed-request states.
- Reproduce the desktop layout; use stacked cards on mobile.

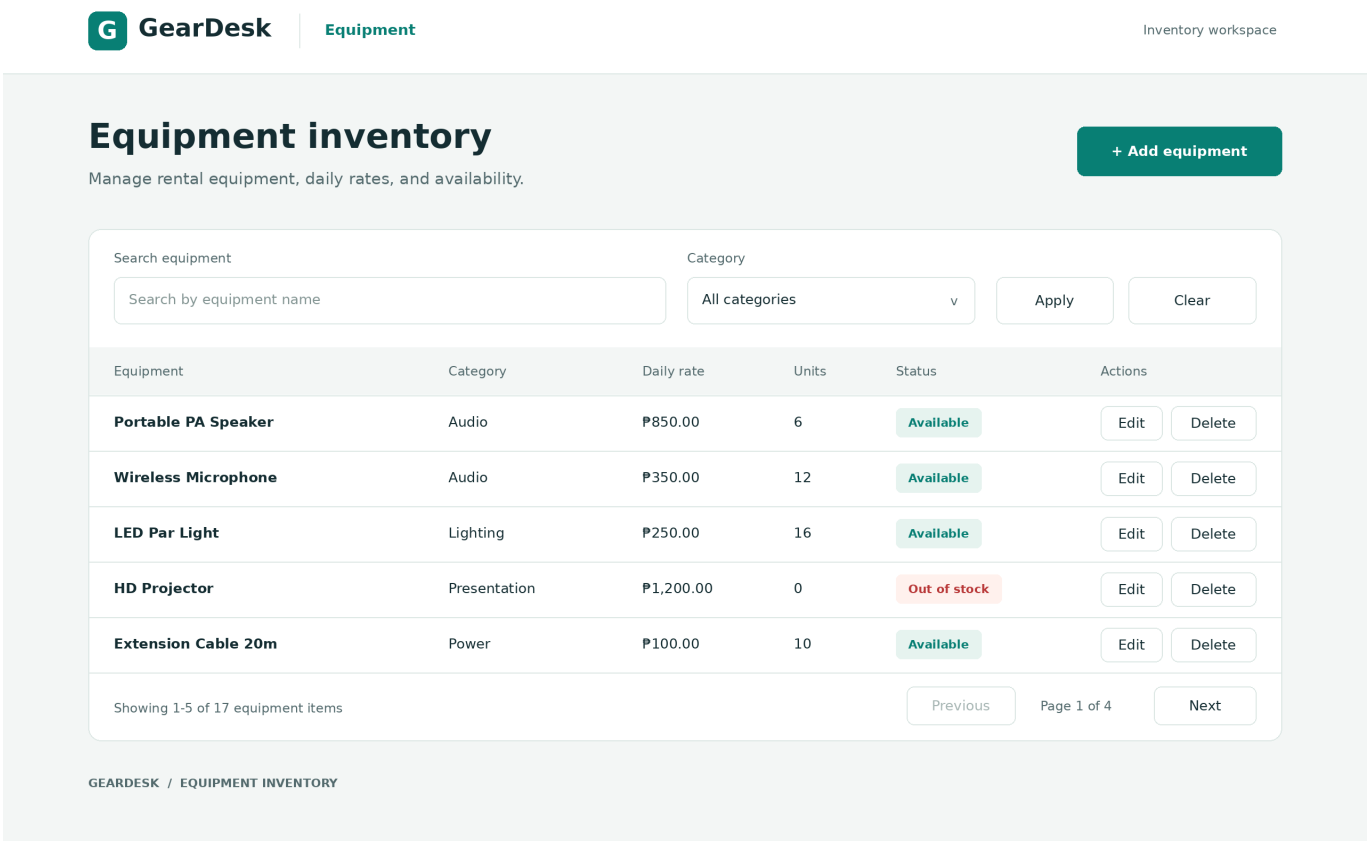
Your constraint

Consume the provided API. Do not build a replacement backend, connect Vue directly to MySQL, add authentication, or generate equipment IDs. Database credentials stay in the backend .env.

Read CLIENT-REQUIREMENTS.md for the complete task and API-DOCUMENTATION.md for the exact contract. The supplied UI images are reference targets, not frontend source code.

One clear inventory workspace.

Target viewport: 1280px. Reference colors, spacing, hierarchy, and controls.



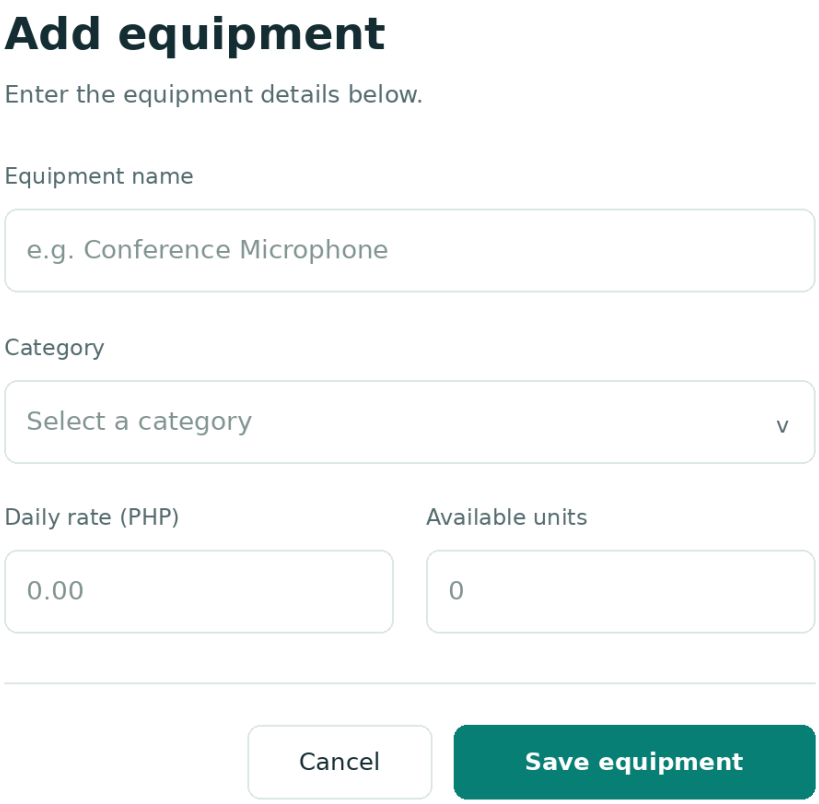
What to reproduce

Region	Required behavior
Header	Brand, Equipment navigation, inventory title, subtitle, Add equipment action.
Toolbar	Name search, all-category filter, Apply, Clear. Both filters reset page to 1.
Table	Equipment, category, daily rate, available units, derived status, Edit/Delete.
Footer	Current range and total, Previous, Page X of Y, Next.
Visual system	Navy text #142D32, teal #087F74, background #F3F6F5; white panels.

On the fresh database, page 1 shows IDs 1-5. There are 17 records and 4 pages. Status is derived: zero units means Out of stock. All other records show Available.

Forms that finish the task.

Use one Add modal, one Edit modal, and one Delete confirmation outside record loops.

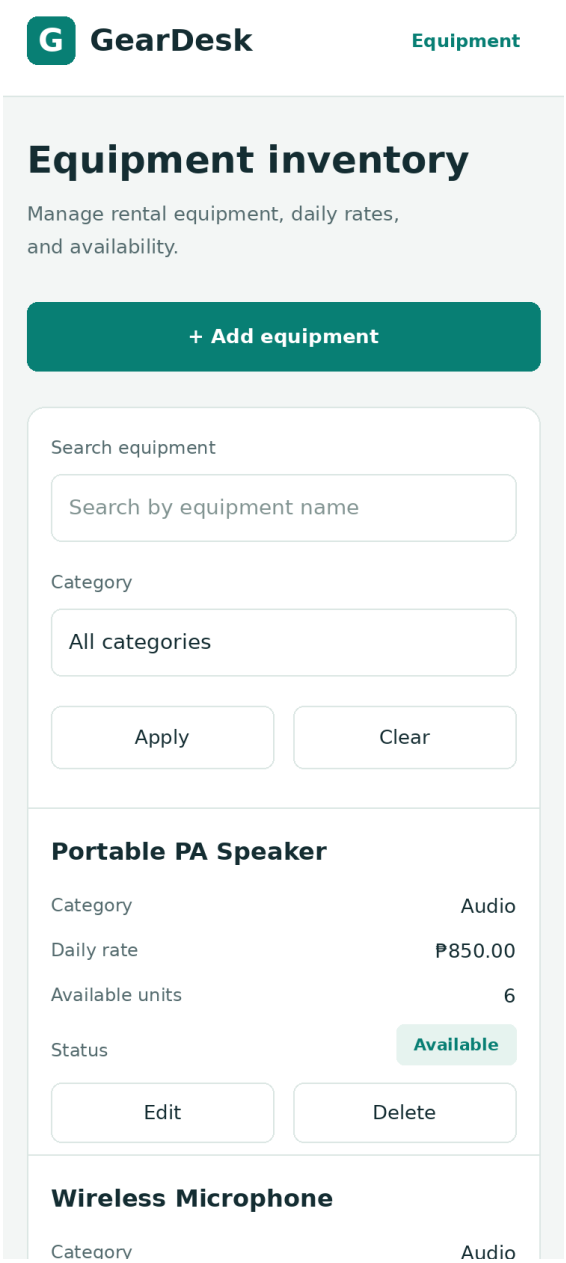


Field	Rule
Equipment name	Trimmed, required, 1-80 characters.
Category	Numeric ID selected from the category API.
Daily rate	Number above 0; maximum 999999.99; at most 2 decimal places.
Available units	Whole number from 0 to 9999. Zero is valid.

Cancel must not mutate the original record. Close the form only on success. Preserve inputs and show field errors after failed requests. Disable repeated saves while pending. Edit must support all four fields.

The same task on a phone.

Target widths: 375px and 320px. Full-length mobile.png is included in mockups/.



Stack the layout

Place Add equipment below the title. Let search and filter controls wrap.

Use equipment cards

Each card keeps the name, category, rate, units, status, Edit, and Delete.

Keep actions reachable

Use readable text and touch-friendly buttons. Avoid page-wide horizontal scrolling.

Manage modal focus

Focus enters the dialog, stays inside while open, and returns to its trigger when closed.

Show failures clearly

Status text must remain visible even with an empty list. Place form errors inside the modal.

Use semantic labels and buttons, visible keyboard focus, accessible dialog titles, Escape to close when idle, and plain-text rendering for user input.

Read the response shape.

Base: http://localhost/geardesk-api/index.php (change Apache port if needed).

Method	Query after base URL	Success
GET	?resource=health	200
GET	?resource=categories	200
GET	?resource=equipment&page=1&per_page=5	200
GET	?resource=equipment&id=1	200
POST	?resource=equipment	201
PATCH	?resource=equipment&id=1	200
DELETE	?resource=equipment&id=1	204, no body

List responses are wrapped

data contains only the requested page. **meta** contains page, per_page, total, and total_pages. On a fresh database: total 17, total_pages 4. Do not slice the five returned records again.

Writable fields

name, category_id, daily_rate, available_units. POST requires all four. PATCH accepts one or more. Send Content-Type: application/json; numeric fields must be JSON numbers. id and category_name are read-only.

Search and category filter

Optional query parameters: search (name substring) and category_id (numeric ID). Combine with AND; URL-encode values. Results sort by ascending ID. Category choices come from the separate categories endpoint.

Failure responses

400 malformed request; 404 missing record/resource; 405 wrong method; 415 wrong content type; 422 field validation; 503 database unavailable. A 422 response has message and errors. DELETE success has no JSON body.

The complete examples, edge-case rules, CORS settings, and seed checkpoints are in API-DOCUMENTATION.md. No authentication or token is required in this local exercise.

Connect before you build.

Complete this preparation before starting the five-hour implementation timer.

1. Start XAMPP

Start Apache and MySQL from XAMPP Control Panel.

2. Copy the backend

Copy xampp/geardesk-api into C:/xampp/htdocs/. Confirm index.php is directly inside geardesk-api.

3. Import the database

Open http://localhost/phpmyadmin/, choose Import, and select database/geardesk.sql. It creates geardesk_practice with 17 equipment records and 4 categories.

4. Configure backend credentials

Edit the backend .env. Defaults: host 127.0.0.1, MySQL port 3306, database geardesk_practice, username root, empty password. Use your actual local MySQL password if configured.

5. Check the API

Open the health URL, then categories and equipment. Health must show database: connected. Run verify-api.ps1 if script execution is permitted on your machine.

6. Configure the frontend

Create Vue separately. Copy frontend.env.example to .env.local. It contains only VITE_API_BASE_URL. Restart Vite after editing it.

7. Begin the timed attempt

Read the client brief, write a short plan, then build the interface. Keep Apache and MySQL running.

If Apache uses port 8080, include :8080 in all HTTP URLs. If Vite uses another origin, add the exact origin to backend ALLOWED_ORIGINS. Do not put database secrets in VITE_* values.

Prove the complete workflow.

Complete MANUAL-TESTS.csv with actual observed results; do not mark untested checks as passing.

- Add, edit all four fields, and delete; refresh to confirm persistence.
- Cancel edit and delete without changing data.
- Test zero units, negative/fractional units, blank names, and invalid rates.
- Navigate all pages; filter/search; test no matches and clear filters.
- Delete all records on the final page and return to a valid page.
- Stop Apache or MySQL; confirm visible errors, preserved form input, and successful Retry.
- Test keyboard access, modal focus, plain-text names, and both target screen widths.
- Run the frontend production build and check for unresolved browser errors.

Practice rubric - 100 points

Area	Points
Mockup fidelity, mobile layout, accessibility	20
API data and category loading	10
Add/edit all fields	20
Delete and confirmation	10
Server pagination, search, category filter	20
Validation and failed-request behavior	10
Plan, Git, tests, README, successful build	10

Submit

Frontend source, package files, public .env.example, README, short plan, completed test sheet, and meaningful Git checkpoints. Exclude node_modules and real backend secrets.

Training target: 80/100 with CRUD and pagination working. This rubric is not official TESDA scoring. Verification limits for the supplied backend are recorded in VERIFICATION.md.